

第13週: データの取得 (Next.js Data Fetching)

～ サーバーサイドでデータを準備する ～

1. Server Components でのデータ取得

Next.js の最大の特徴は、**サーバー側で API を叩いてから画面を表示できる**ことです。

- **メリット:**

- ブラウザ側で JS を動かす必要がないため、表示が速い。
- セキュリティが高い（APIキーを隠せる）。
- SEO に強い。

2. `async / await` を Component で使う

React Server Components では、関数自体を `async` にできます。

```
// app/page.tsx (デフォルトは Server Component)
export default async function Page() {
  const res = await fetch('https://api.github.com/users/octocat');
  const user = await res.json();

  return <h1>{user.name}</h1>;
}
```

3. Server vs Client の使い分け

- **Server Component (デフォルト)**: データの取得、SEO、重い処理。
- **Client Component** (`'use client'`): ボタンクリック、フォーム入力、`useState` などの「動き」。

黄金パターン: サーバー側でデータを取得し、それをクライアントコンポーネントに Props として渡す。

4. 本日の目標: データの完全移行

GitHub API や 天気 API を使って、ダッシュボードを本物のデータで満たします。

- ユーザー情報をサーバーサイドで取得。
- コンポーネントにデータを流し込む。
- 「読み込み中」の表示を `loading.tsx` でスマートに実装する。

5. Phase 3 完了！

これで、Vanilla JS 版のダッシュボードが、最新の Next.js アプリへと完全に生まれ変わりました。

- 第10週: 構造のリプレイス (Next.js & Tailwind)
- 第11週: コンポーネント分割
- 第12週: 状態管理 (useState)
- 第13週: データ取得 (Server Components)

宿題 (Homework)

1. `fetch` を使って、自分の GitHub 情報をダッシュボードに表示する。
2. 天気やニュースなど、外部の API を一つ以上統合する。
3. `loading.tsx` ファイルを作成し、データ取得中の見栄えを整える。